Handwritten Digit Classification System Powered by FPGA

Group - 7 Design Choices & Results

Convolution Architecture

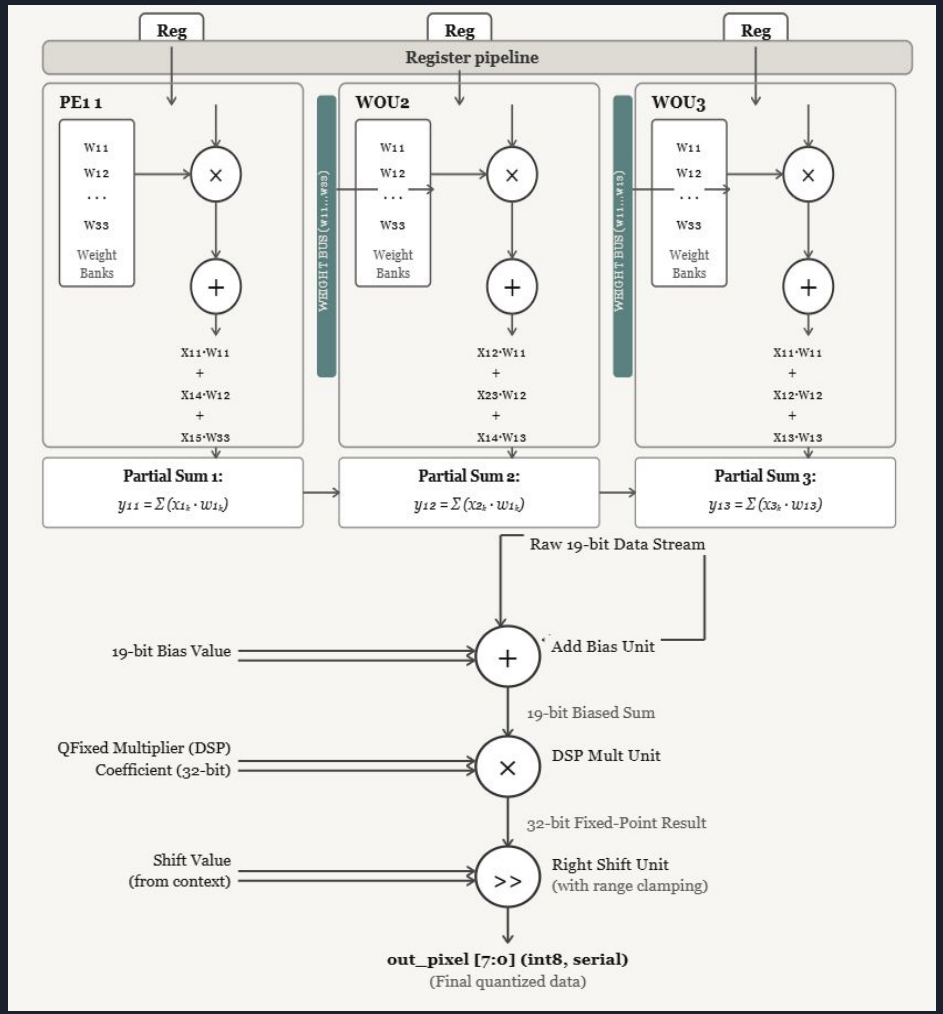
LUTs	669
FFs	551
BRAMs	0
DSPs	4

Design Trade Offs

Area: Precision-matched bit-widths (19-bit acc, 11-bit clamp) + LUT multipliers over DSPs → **669 LUTs**

Performance: 1-D PE processes one kernel row per pass **3 cycles/pixel**, accepted since MNIST inference remains **< 1ms**

Power: Narrow datapaths + LUT MACs over DSP48s → reduced switching activity, **0 BRAM** leakage



Ref: Li, Huimin, et al. "A high performance FPGA-based accelerator for large-scale convolutional neural networks." *2016 26th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2016.

Fully connect Architecture

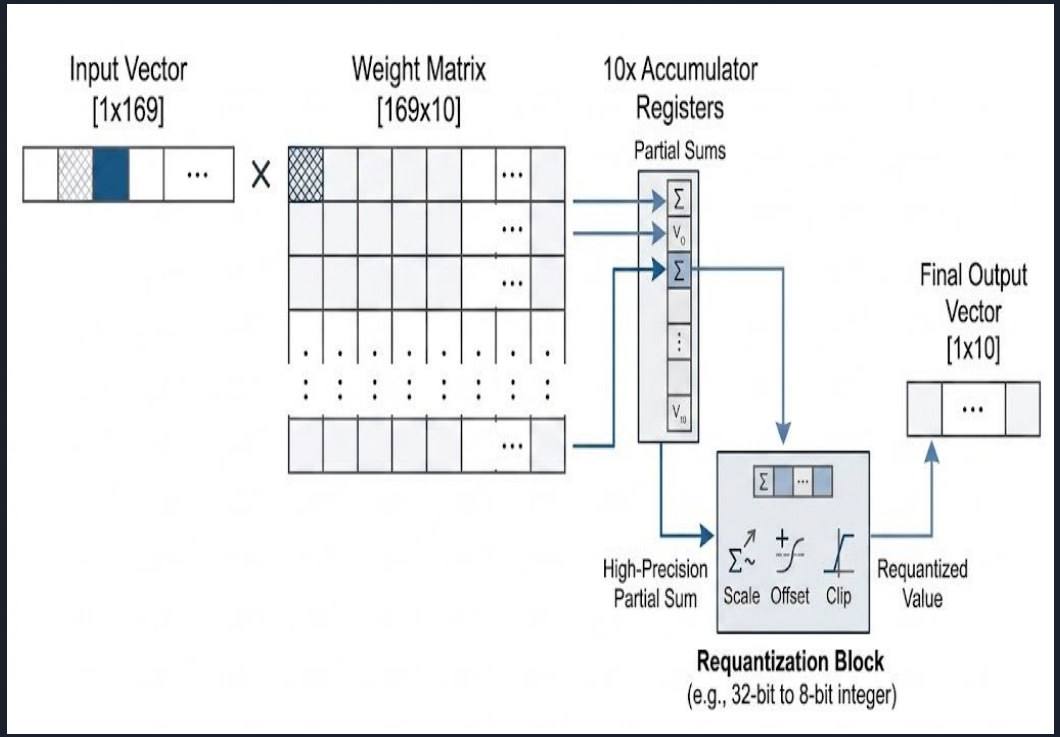
LUTs	528
FFs	507
BRAMs	0.5
DSPs	2

<u>Design Trade Offs</u>

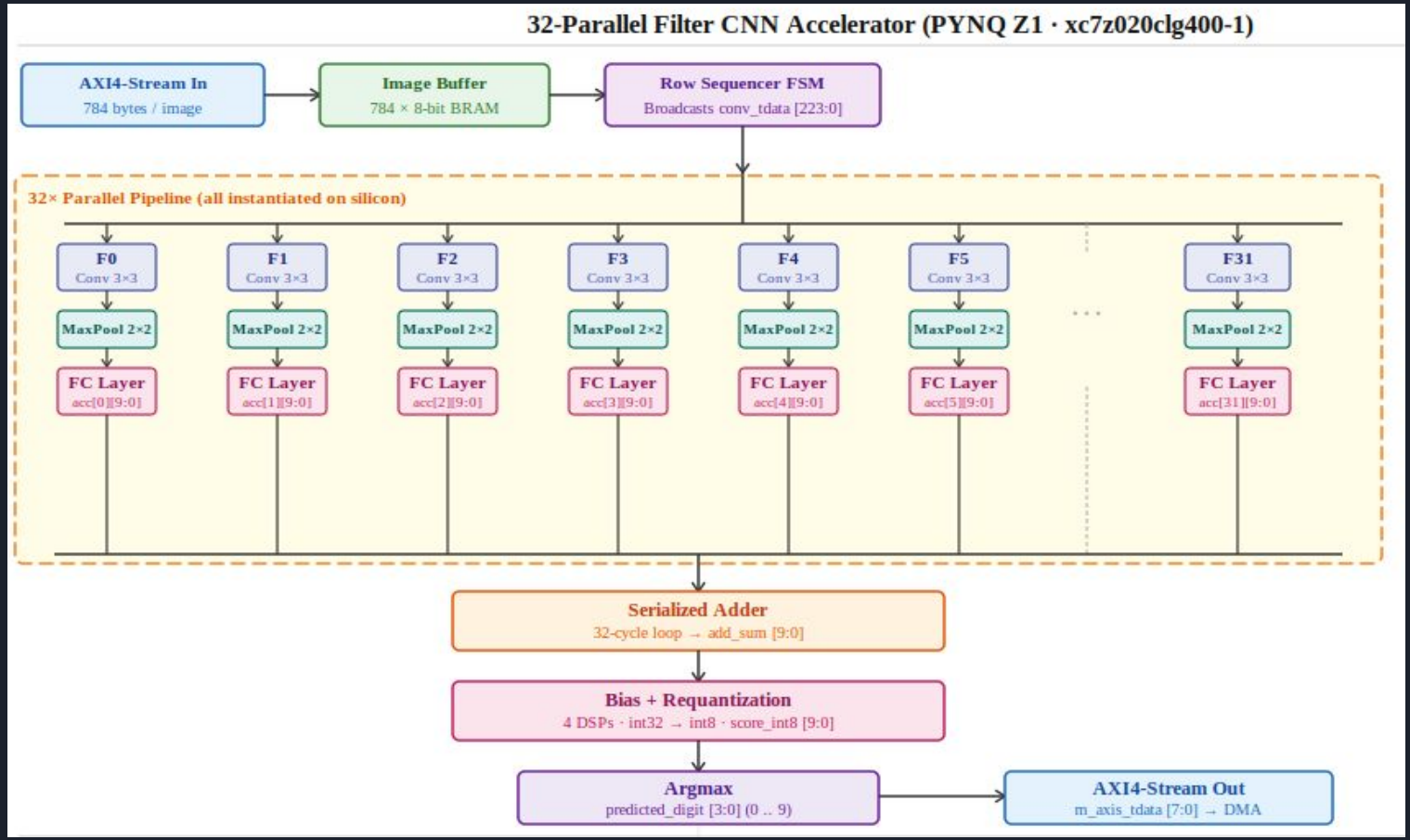
Area: 2 DSPs, dual BRAM ports

Performance: 2 DSP multipliers compute 2 outputs per cycle , processing 5 pairs across 13 columns **sequential but fully pipelined**; latency accepted since FC layer is post-pooling on a 13×13 feature map

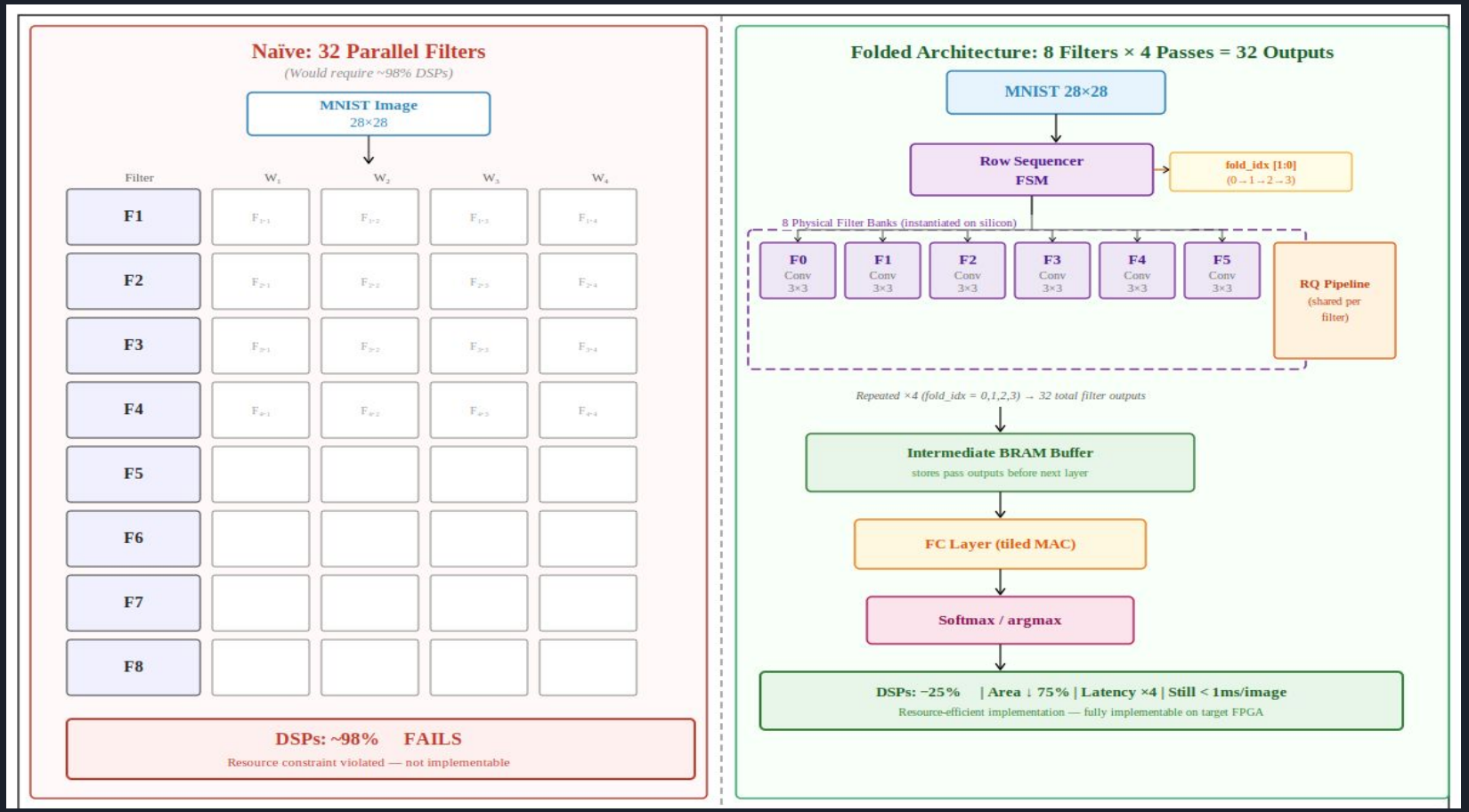
Power: Weights stored in **block RAM** (not registers) eliminates switching activity of 1,690 flip-flops



Parallel Architecture



Folded Architecture



Results - PYNQ Z1

Fully Parallel Architecture Synthesis

LUTs	4872	92%
FFs	6773	64%
BRAMs	16	11%
DSPs	216	98%
IOs	35	28%
BUFG	1	3.5%

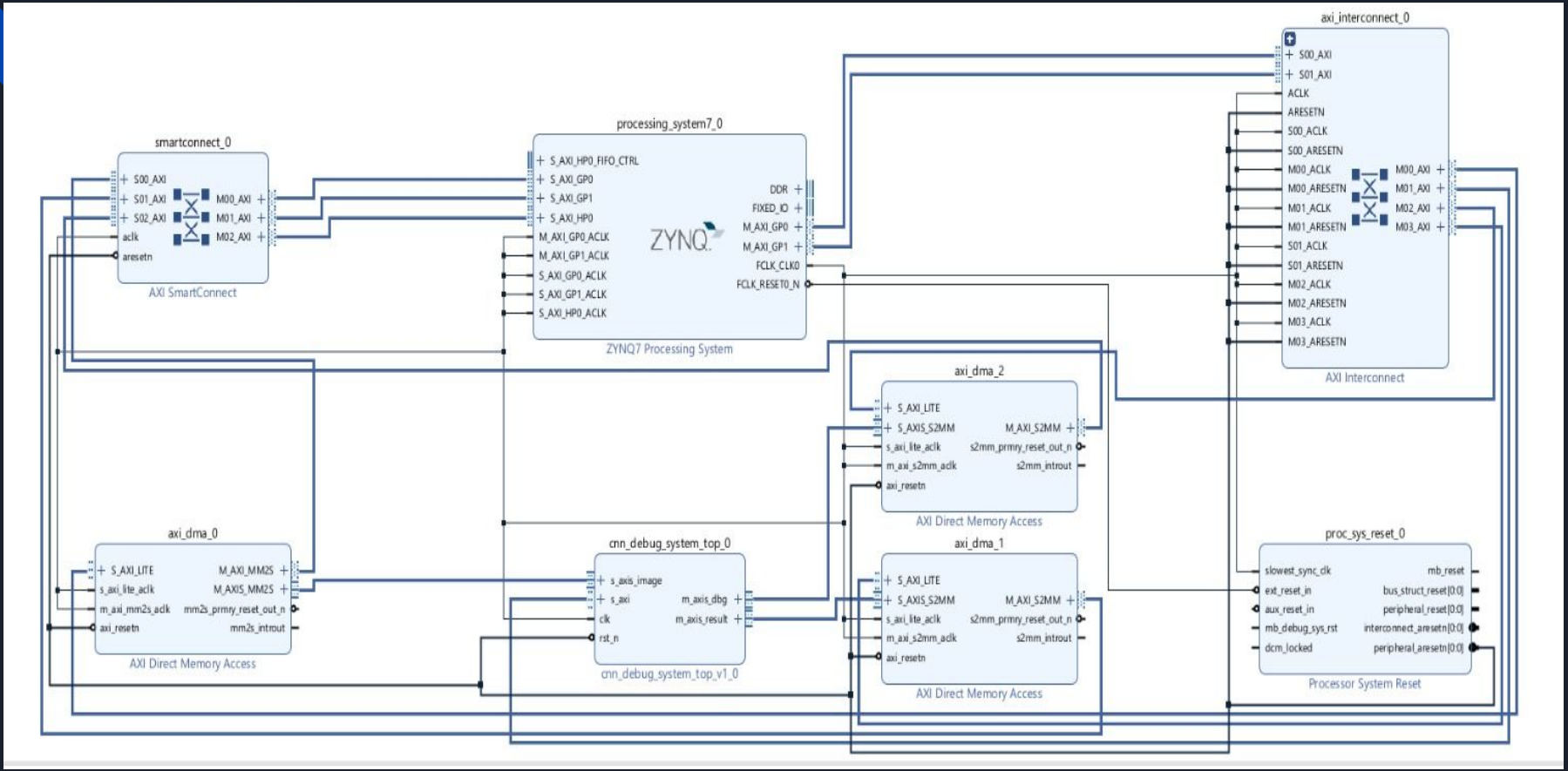
Failed Implementation !!!

Folded Architecture Implementation

LUTs	12688	23.85%
FFs	23609	22.19%
BRAMs	16.5	11.79%
DSPs	61	27.73%
IOs	35	28%
BUFG	1	3.13%

Performance: 41.6MHz
Total On chip Power: 0.156W

Grade 3 Design



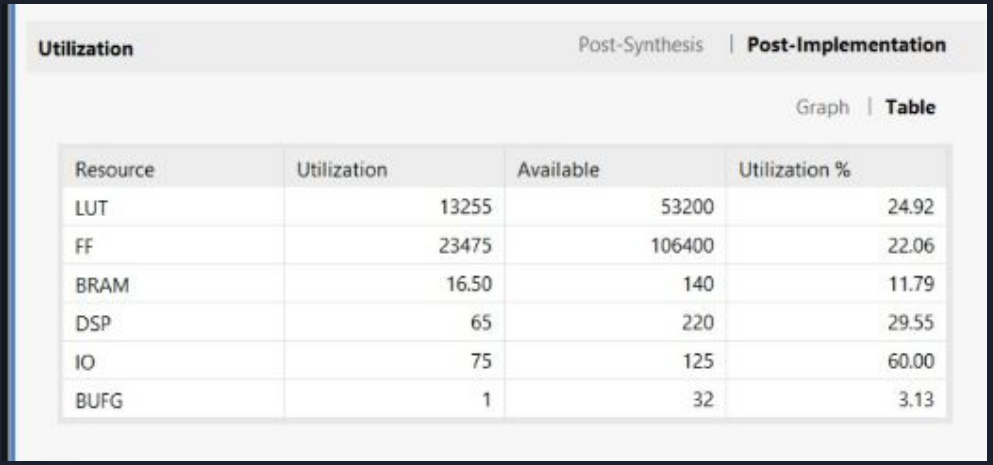
Results - PYNQ Z1

Folded CNN ip alone

LUTs	12688	23.85%
FFs	23609	22.19%
BRAMs	16.5	11.79%
DSPs	61	27.73%
IOs	35	28%
BUFG	1	3.13%

Performance: 41.6MHz
Total On chip Power: 0.156W

Folded Architecture Implementation



Utilization Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	13255	53200	24.92
FF	23475	106400	22.06
BRAM	16.50	140	11.79
DSP	65	220	29.55
IO	75	125	60.00
BUFG	1	32	3.13

Grade 3 Results

```
print(f'\nFinal: {correct}/{total}
```

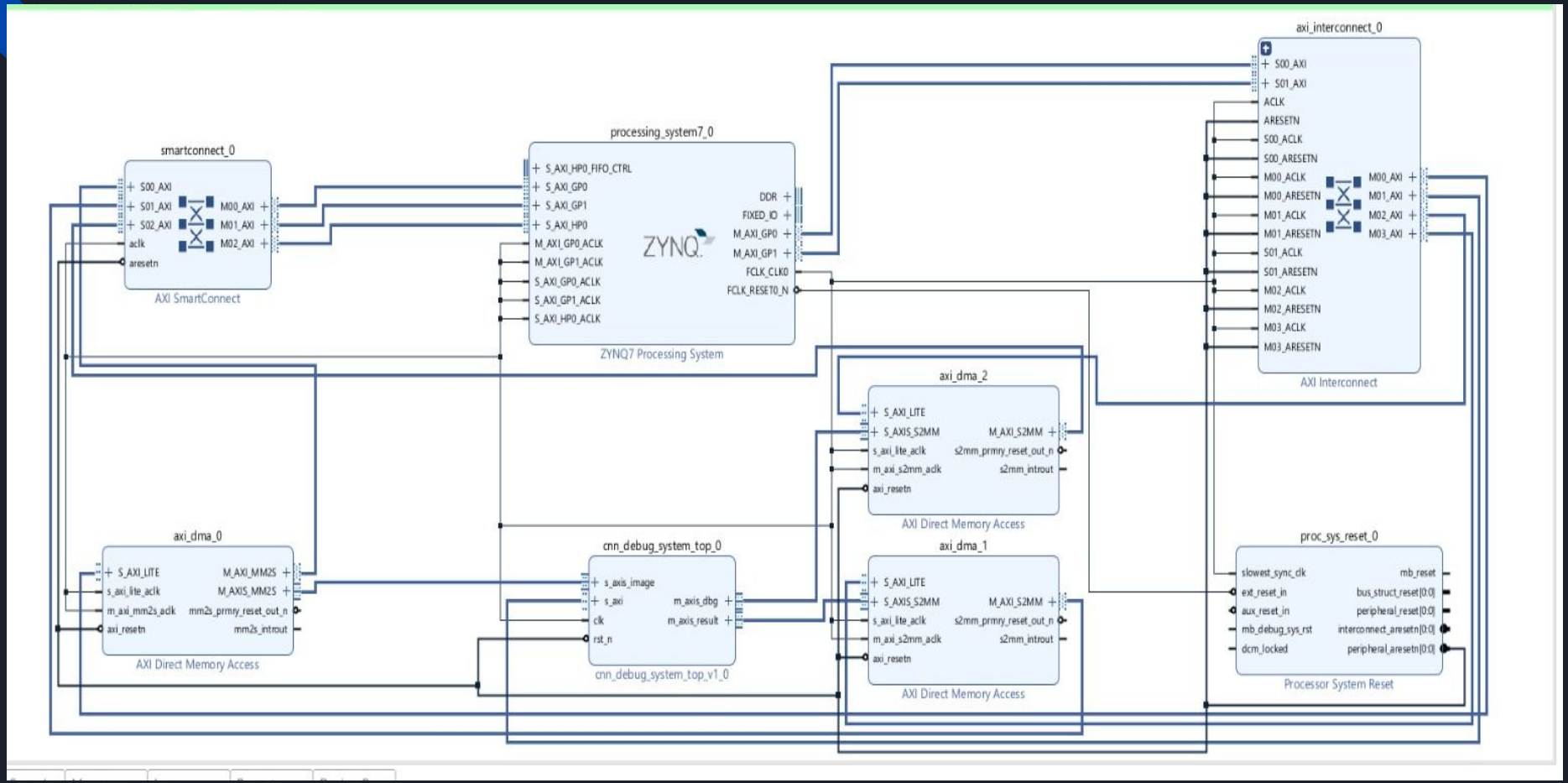
```
[ 100/1000] Accuracy: 97.0%
[ 200/1000] Accuracy: 96.5%
[ 300/1000] Accuracy: 96.3%
[ 400/1000] Accuracy: 96.0%
[ 500/1000] Accuracy: 96.0%
[ 600/1000] Accuracy: 96.3%
[ 700/1000] Accuracy: 96.3%
[ 800/1000] Accuracy: 96.4%
[ 900/1000] Accuracy: 96.2%
[1000/1000] Accuracy: 96.2%

Final: 962/1000 = 96.20%
```

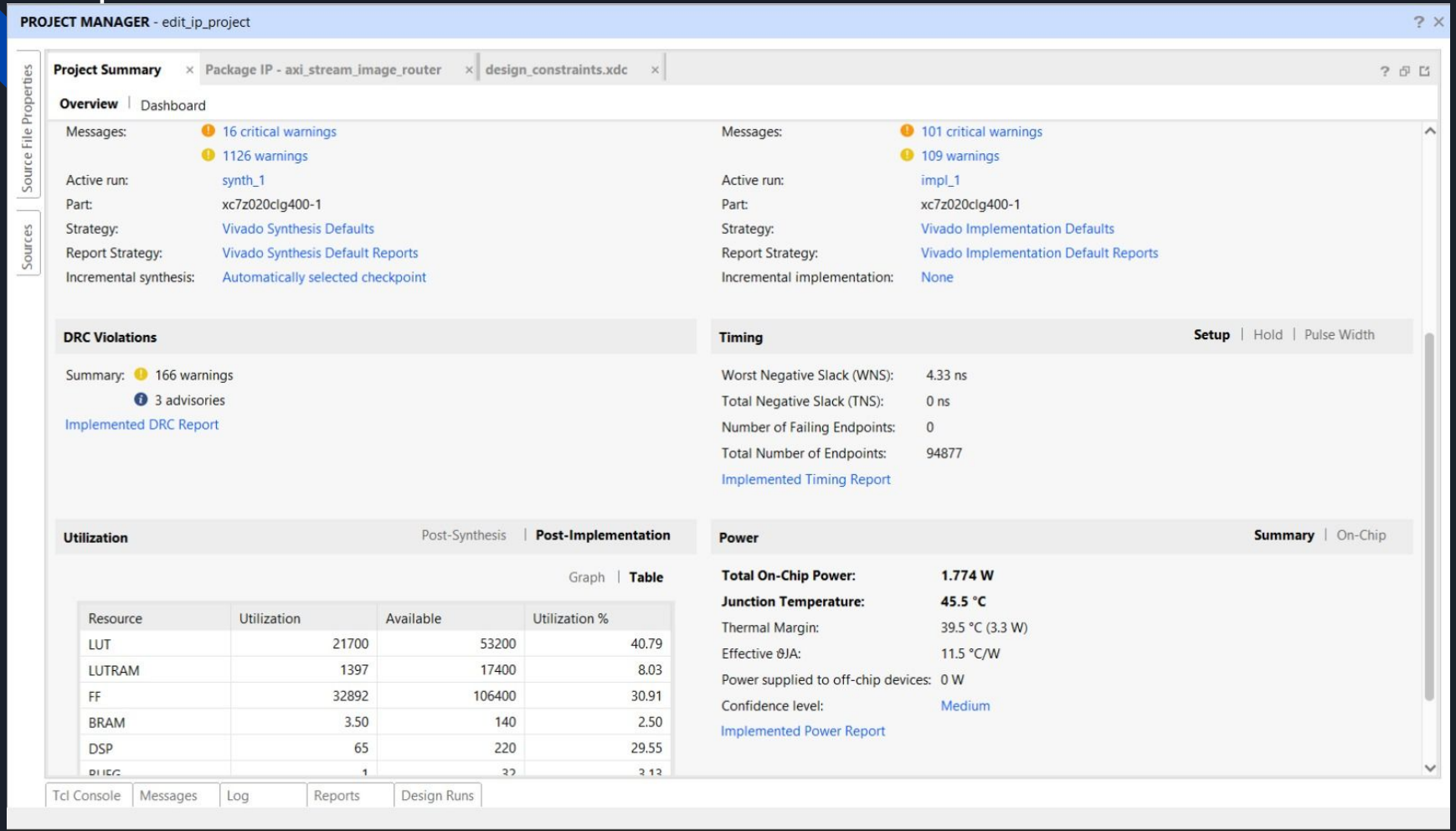
```
Loading bitstream...
Done

============================================
  CNN Inference — Custom MNIST Images
============================================
  Image                    True  Pred  Result
  ----------------------------------------
  mnist_0_28x28.png           0     0      OK
  mnist_2_28x28.png           2     2      OK
  mnist_5_28x28.png           5     5      OK
  mnist_7_28x28.png           7     7      OK
  mnist_9_28x28.png           9     9      OK
  ----------------------------------------
  Accuracy: 5/5 = 100%
============================================
```

Grade 4 Design



Implementation results



Pynq Results

```
============================================================
 Sending batch of 10 image(s)
============================================================

[1] Router configured:
  CTRL       = 0x00000001  (enable=1, soft_reset=0)
  BATCH_SIZE = 10
  STATUS     = STREAM_IMG (state=1, busy=1)
  BEATS      = 0
  IMG_DONE   = 0
  ERR        = early_tlast=0, bp_timeout=0

[2] Sending 10 x 784 = 7840 bytes via DMA0 MM2S...
[3] Waiting for 10 result byte(s)...

============================================================
 INFERENCE RESULTS  (0.01s for 10 image(s))
============================================================
  Image   0: predicted = 7  true = 7  [OK]
  Image   1: predicted = 2  true = 2  [OK]
  Image   2: predicted = 1  true = 1  [OK]
  Image   3: predicted = 0  true = 0  [OK]
  Image   4: predicted = 4  true = 4  [OK]
  Image   5: predicted = 1  true = 1  [OK]
  Image   6: predicted = 4  true = 4  [OK]
  Image   7: predicted = 9  true = 9  [OK]
  Image   8: predicted = 5  true = 5  [OK]
  Image   9: predicted = 9  true = 9  [OK]

  Accuracy: 10/10 = 100.0%
```

```
[1] Router configured:
  CTRL       = 0x00000001  (enable=1, soft_reset=0)
  BATCH_SIZE = 5
  STATUS     = STREAM_IMG (state=1, busy=1)
  BEATS      = 0
  IMG_DONE   = 0
  ERR        = early_tlast=0, bp_timeout=0

[2] Sending 5 x 784 = 3920 bytes via DMA0 MM2S...
[3] Waiting for 5 result byte(s)...

============================================================
 INFERENCE RESULTS  (0.01s for 5 image(s))
============================================================
  Image   0: predicted = 7  true = 7  [OK]
  Image   1: predicted = 2  true = 2  [OK]
  Image   2: predicted = 1  true = 1  [OK]
  Image   3: predicted = 0  true = 0  [OK]
  Image   4: predicted = 4  true = 4  [OK]

  Accuracy: 5/5 = 100.0%

Router status (expect SEND_TLAST stuck — known RTL issue):
  CTRL       = 0x00000001  (enable=1, soft_reset=0)
  BATCH_SIZE = 5
  STATUS     = STREAM_IMG (state=1, busy=1)
  BEATS      = 3920
  IMG_DONE   = 0
  ERR        = early_tlast=0, bp_timeout=0
  labels:     [7, 2, 1, 0, 4]
  predicted:  [7, 2, 1, 0, 4]
```

Pynq Results

```
Batch size = 5  (1000 images)
  [ 200/1000]  correct= 193  acc=96.5%  elapsed=0.7s
  [ 400/1000]  correct= 384  acc=96.0%  elapsed=1.5s
  [ 600/1000]  correct= 578  acc=96.3%  elapsed=2.2s
  [ 800/1000]  correct= 771  acc=96.4%  elapsed=2.9s
  [1000/1000]  correct= 962  acc=96.2%  elapsed=3.7s
  -> Accuracy: 962/1000 = 96.20%  Time: 3.7s  Throughput: 272.2 img/s

Batch size = 10  (1000 images)
  [ 200/1000]  correct= 193  acc=96.5%  elapsed=0.5s
  [ 400/1000]  correct= 384  acc=96.0%  elapsed=1.0s
  [ 600/1000]  correct= 578  acc=96.3%  elapsed=1.6s
  [ 800/1000]  correct= 771  acc=96.4%  elapsed=2.1s
  [1000/1000]  correct= 962  acc=96.2%  elapsed=2.6s
  -> Accuracy: 962/1000 = 96.20%  Time: 2.6s  Throughput: 390.3 img/s

============================================================
  Grade 4 Batch Accuracy  --  1000 images per batch size
============================================================
   Batch |    Correct |  Accuracy |  Time(s) |    img/s
  -----------------------------------------------------------
       1 |   962/1000 |    96.20% |     16.4 |     60.8
       3 |   962/1000 |    96.20% |      5.8 |    173.0
       5 |   962/1000 |    96.20% |      3.7 |    272.2
      10 |   962/1000 |    96.20% |      2.6 |    390.3
============================================================
```

Debugging Results

DMA2 Raw Byte Capture
Output came back as collapsed 3×2×2 shape instead of expected 26×26 / 13×13 / 169×32

Live Board DMA2 vs Python Comparison
All predictions pointed to digit 1 regardless of input image
Digit 1 has largest bias (4022) — revealed FC weights were loading as zeros

Structured .txt File Comparison
RTL testbench dumps all stage outputs to structured .txt file
Python behavioural model generates matching structured .txt file
compares stage by stage with per-filter visibility

Grade 5